

Архитектура Компьютеров и
операционные системы

Лабораторная работа №13

**Программирование в командном
процессоре ОС UNIX.
Ветвления и циклы**

ОТЧЁТ

МВЕТУВОП ФОНГАНГ ДЖУЛС ФЕРРИ
1132255560

Цель работы

Изучить основы программирования в оболочке ОС UNIX.
Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задание №1. Скрипт с `getopts` для поиска строк

Краткое описание: Я написал скрипт, который анализирует опции командной строки. Этот скрипт принимает входной файл, выходной файл, шаблон поиска, может учитывать или не учитывать регистр букв, а также выводить номера строк.

Результат :

Скрипт работает правильно. Он находит все строки, соответствующие шаблону, и выводит их на экран или сохраняет в файл в зависимости от выбранной опции.

script1.sh	test.txt	resultat.txt
<pre>1 #!/bin/bash 2 3 # Initialisation des variables 4 input_file="" 5 output_file="" 6 pattern="" 7 case_sensitive="" 8 show_numbers="" 9 10 # Afficher l'aide 11 usage() { 12 echo "Usage: \$0 -i <inputfile> -o <outputfile> -p <pattern> [-C] [-n]" 13 echo " -i : fichier d'entrée" 14 echo " -o : fichier de sortie" 15 echo " -p : motif de recherche" 16 echo " -C : sensible à la casse (par défaut, insensible)" 17 echo " -n : afficher les numéros de ligne" 18 exit 1 19 } 20 21 # Analyse des options avec getopt 22 while getopt "i:o:p:Cn" opt; do 23 case \$opt in 24 i) input_file="\$OPTARG" ;; 25 o) output_file="\$OPTARG" ;; 26 p) pattern="\$OPTARG" ;; 27 C) case_sensitive="C" ;; 28 n) show_numbers="n" ;; 29 *) usage ;; 30 esac 31 done 32 33 # Vérifier les paramètres obligatoires 34 if [-z "\$input_file"] [-z "\$pattern"]; then 35 echo "Erreur: Les options -i et -p sont obligatoires"</pre>		

```
fdmvetuvop@jules-N-A:/home/fdmvetuvop/Bureau/LAB 13
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ echo "Ligne 1: Bonjour le monde" > test.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ echo "Ligne 2: Hello World" >> test.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ echo "Ligne 3: BONJOUR encore" >> test.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ echo "Ligne 4: Programmation shell" >> test.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ echo "Ligne 5: bonjour bash" >> test.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ chmod +x script1.sh
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script1.sh -i test.txt -p "bonjour"
Ligne 1: Bonjour le monde
Ligne 3: BONJOUR encore
Ligne 5: bonjour bash
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script1.sh -i test.txt -p "bonjour" -C
Ligne 5: bonjour bash
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script1.sh -i test.txt -p "hello" -n
2:Ligne 2: Hello World
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script1.sh -i test.txt -p "bonjour" -o resultat.txt
Résultats écrits dans : resultat.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ cat resultat.txt
Ligne 1: Bonjour le monde
Ligne 3: BONJOUR encore
Ligne 5: bonjour bash
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$
```

Задание №2. Программа на С и скрипт оболочки для анализа кода возврата

Краткое описание: Я написал небольшую программу на языке С, которая запрашивает у пользователя число. В зависимости от того, является ли число положительным, отрицательным или нулём, программа завершается с разным кодом возврата (1, 2 или 0). Скрипт оболочки вызывает эту программу, получает код возврата через переменную \$? и выводит соответствующее сообщение.

Результат :

Скрипт правильно определяет, является ли введённое число нулём, положительным или отрицательным, анализируя код возврата программы на С.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int nombre;
6
7     printf("Entrez un nombre entier : ");
8     scanf("%d", &nombre);
9
10    if (nombre > 0) {
11        printf("Vous avez saisi un nombre positif : %d\n", nombre);
12        exit(1);
13    } else if (nombre < 0) {
14        printf("Vous avez saisi un nombre negatif : %d\n", nombre);
15        exit(2);
16    } else {
17        printf("Vous avez saisi zero\n");
18        exit(0);
19    }
20
21    return 0;
22 }
```

```
1 #!/bin/bash
2
3 # Compiler le programme C
4 echo "Compilation du programme C..."
5 gcc -o nombre nombre.c
6
7 if [ $? -ne 0 ]; then
8     echo "Erreur de compilation"
9     exit 1
10 fi
11
12 echo "Compilation reussie"
13 echo ""
14
15 # Exécuter le programme C
16 ./nombre
17
18 # Analyser le code de retour
19 code_retour=$?
20
21 echo ""
22 echo "=== Analyse du code de retour ==="
23 echo "Code de retour : $code_retour"
24
25 case $code_retour in
26     0) echo "Message: Le nombre saisi est ZERO" ;;
27     1) echo "Message: Le nombre saisi est POSITIF" ;;
28     2) echo "Message: Le nombre saisi est NEGATIF" ;;
29     *) echo "Message: Code de retour inconnu" ;;
30 esac
```

```
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ chmod +x script2.sh
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script2.sh
Compilation du programme C...
Compilation reussie

Entrez un nombre entier : 4
Vous avez saisi un nombre positif : 4

=== Analyse du code de retour ===
Code de retour : 1
Message: Le nombre saisi est POSITIF
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$
```

Задание №3. Создание и удаление нумерованных файлов

Краткое объяснение :

Я создал скрипт, который генерирует N файлов с именами 1.tmp, 2.tmp, ..., N.tmp во временной папке. Скрипт также умеет удалять все эти файлы с помощью опции `–delete`.

Результат :

Скрипт создаёт ровно столько файлов, сколько было запрошено, и правильно удаляет их при использовании опции `--delete`.

```

1 #!/bin/bash
2
3 # Vérifier le nombre d'arguments
4 if [ $# -lt 1 ]; then
5     echo "Usage: $0 <N> [--delete]"
6     echo "  N : nombre de fichiers à créer (entre 1 et 100)"
7     echo "  --delete : supprimer tous les fichiers créés"
8     exit 1
9 fi
10
11 # Répertoire pour les fichiers
12 DIR="$HOME/tmp_files"
13 mkdir -p "$DIR"
14
15 # Fonction pour créer les fichiers
16 create_files() {
17     local N=$1
18     echo "Création de $N fichiers dans $DIR/"
19
20     for i in $(seq 1 $N); do
21         touch "$DIR/$i.tmp"
22         echo "  - Créé : $i.tmp"
23     done
24
25     echo "Terminé ! $N fichiers créés."
26     ls -la "$DIR"/*.tmp 2>/dev/null | wc -l | xargs echo "Nombre total de fichiers .tmp :"
27 }
28
29 # Fonction pour supprimer les fichiers
30 delete_files() {
31     echo "Suppression de tous les fichiers .tmp dans $DIR/"
32
33     count=$(ls "$DIR"/*.tmp 2>/dev/null | wc -l)
34
35     if [ $count -eq 0 ]; then

```

sh ▾ Largeur des tabulations : 8 ▾

```

fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ chmod +x script3.sh
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script3.sh 5
Suppression de tous les fichiers .tmp dans /home/fdmvetuvop/tmp_files/
Aucun fichier .tmp à supprimer.
Création de 5 fichiers dans /home/fdmvetuvop/tmp_files/
  - Créé : 1.tmp
  - Créé : 2.tmp
  - Créé : 3.tmp
  - Créé : 4.tmp
  - Créé : 5.tmp
Terminé ! 5 fichiers créés.
Nombre total de fichiers .tmp : 5

Contenu du répertoire /home/fdmvetuvop/tmp_files/:
total 8
drwxrwxr-x  2 fdmvetuvop fdmvetuvop 4096 mai   9 00:05 .
drwxr-xr-x 31 fdmvetuvop fdmvetuvop 4096 mai   9 00:05 ..
-rw-rw-r--  1 fdmvetuvop fdmvetuvop   0 mai   9 00:05 1.tmp
-rw-rw-r--  1 fdmvetuvop fdmvetuvop   0 mai   9 00:05 2.tmp
-rw-rw-r--  1 fdmvetuvop fdmvetuvop   0 mai   9 00:05 3.tmp
-rw-rw-r--  1 fdmvetuvop fdmvetuvop   0 mai   9 00:05 4.tmp
-rw-rw-r--  1 fdmvetuvop fdmvetuvop   0 mai   9 00:05 5.tmp
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script3.sh --delete
Erreur: N doit être un nombre entre 1 et 100
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$

```

Задание №4. Архивация tar с фильтрацией по дате изменения

Краткое описание: Я написал скрипт, который архивирует все файлы в указанной директории с помощью команды tar. Модифицированная версия использует команду find для архивации только тех файлов, которые были изменены менее недели назад.

Результат :

Скрипт создаёт архив со всеми файлами и отдельный архив только с недавно изменёнными файлами (за последние 7 дней).


```

1 #!/bin/bash
2
3 REP="$1"
4
5 if [ $# -lt 1 ]; then
6     echo "Usage: $0 <repertoire>"
7     exit 1
8 fi
9
10 if [ ! -d "$REP" ]; then
11     echo "Erreur: Le répertoire '$REP' n'existe pas"
12     exit 1
13 fi
14
15 DATE=$(date +%Y%m%d_%H%M%S)
16 ARCHIVE="archive_$(date +%Y%m%d_%H%M%S).tar.gz"
17
18 # Version 1: Tous les fichiers
19 echo "=== Version 1: Archivage de tous les fichiers ==="
20 tar -czf "$ARCHIVE" "$REP"
21 echo "Archive créée: $ARCHIVE"
22 ls -lh "$ARCHIVE"
23
24 # Version 2: Fichiers modifiés depuis moins d'une semaine
25 echo ""
26 echo "=== Version 2: Archivage des fichiers récents (moins de 7 jours) ==="
27 DATE=$(date +%Y%m%d_%H%M%S)
28 ARCHIVE_RECENT="archive_recent_$(date +%Y%m%d_%H%M%S).tar.gz"
29
30 # Utiliser find pour lister les fichiers récents et les archiver
31 find "$REP" -type f -mtime -7 -print | tar -czf "$ARCHIVE_RECENT" -T -
32 echo "Archive récente créée: $ARCHIVE_RECENT"
33 ls -lh "$ARCHIVE_RECENT"
34
35 echo ""

```

```

1 #!/bin/bash
2
3 # Vérification des arguments
4 if [ $# -lt 1 ]; then
5     echo "Usage: $0 <repertoire> [--recent]"
6     echo "  --recent : n'archiver que les fichiers modifiés depuis moins d'une semaine"
7     exit 1
8 fi
9
10 REP="$1"
11 FILTER="$2"
12
13 # Vérifier que le répertoire existe
14 if [ ! -d "$REP" ]; then
15     echo "Erreur: Le répertoire '$REP' n'existe pas"
16     exit 1
17 fi
18
19 # Nom de l'archive avec date
20 DATE=$(date +%Y%m%d_%H%M%S)
21 ARCHIVE_NAME="archive_$(basename "$REP")_$DATE.tar.gz"
22
23 echo "=== Archivage du répertoire: $REP ==="
24
25 if [ "$FILTER" = "--recent" ]; then
26     echo "Mode: Fichiers modifiés depuis moins d'une semaine (7 jours)"
27
28     # Trouver les fichiers modifiés depuis moins de 7 jours
29     # Et les archiver avec tar
30     find "$REP" -type f -mtime -7 -print0 | tar -czf "$ARCHIVE_NAME" --null -T -
31
32     # Compter les fichiers archivés
33     NB_FICHIERS=$(find "$REP" -type f -mtime -7 | wc -l)
34     echo "Nombre de fichiers archivés: $NB_FICHIERS"
35

```

sh ▾ Largeur des ta

```

fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ mkdir -p ~/test_archive
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ echo "Fichier ancien" > ~/test_archive/ancien.tx
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ sleep 2
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ echo "Fichier recent" > ~/test_archive/recent.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ touch -d "8 days ago" ~/test_archive/ancien.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ chmod +x script4.sh
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script4.sh ~/test_archive
=== Archivage du répertoire: /home/fdmvetuvop/test_archive ===
Mode: Tous les fichiers
Nombre de fichiers archivés: 3
Archive créée: archive_test_archive_20260509_001641.tar.gz
-rw-rw-r-- 1 fdmvetuvop fdmvetuvop 234 mai  9 00:16 archive_test_archive_20260509_001641.tar.gz

Contenu de l'archive:
test_archive/
test_archive/ancien.tx
test_archive/recent.txt
test_archive/ancien.txt
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ █

```

```
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ chmod +x script4_new.sh
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ ./script4_new.sh ~/test_archive --recent
=== Version 1: Archivage de tous les fichiers ===
tar: Suppression de « / » au début des noms des membres
Archive créée: archive_20260509_002404.tar.gz
-rw-rw-r-- 1 fdmvetuvop fdmvetuvop 245 mai  9 00:24 archive_20260509_002404.tar.gz

=== Version 2: Archivage des fichiers récents (moins de 7 jours) ===
tar: Suppression de « / » au début des noms des membres
Archive récente créée: archive_recent_20260509_002404.tar.gz
-rw-rw-r-- 1 fdmvetuvop fdmvetuvop 194 mai  9 00:24 archive_recent_20260509_002404.tar.gz

Statistiques:
  Tous les fichiers: 3 fichiers
  Fichiers récents: 2 fichiers
fdmvetuvop@jules-N-A:~/Bureau/LAB 13$ █
```

Выводы

В ходе выполнения данной лабораторной работы я научился :

- использовать getopt для анализа сложных опций командной строки
- организовать взаимодействие между программой на C и скриптом оболочки через коды возврата
- создавать и удалять файлы в циклах
- фильтровать файлы по дате изменения с помощью команды find перед архивацией с помощью tar

Эти навыки позволяют мне теперь писать более профессиональные и хорошо структурированные скрипты оболочки.

Ответы на контрольные вопросы

1. Каково предназначение команды `getopts`?

Команда `getopts` предназначена для анализа опций и их аргументов в командной строке. Она используется внутри цикла `while` для обработки каждой встреченной опции и позволяет легко управлять такими опциями, как `-i`, `-o`, `-r`, с аргументами или без них.

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы, такие как звёздочка, вопросительный знак и квадратные скобки, позволяют генерировать списки имён файлов, соответствующих определённому шаблону. Командный процессор автоматически заменяет эти шаблоны списком подходящих файлов.

3. Какие операторы управления действиями вы знаете?

Я знаю оператор `if` для условий, `case` для множественного выбора, `for` для циклов по списку, `while` для циклов с предусловием, `until` для циклов с условием окончания, а также `break` и `continue` для управления выполнением циклов.

4. Какие операторы используются для прерывания цикла?

Для прерывания цикла используется оператор `break`, который полностью завершает выполнение цикла, и оператор `continue`, который прерывает текущую итерацию и переходит к следующей, не завершая цикл полностью.

5. Для чего нужны команды `false` и `true`?

Команда `true` всегда возвращает ноль (истина), а команда `false` всегда возвращает ненулевой код (ложь). Они

используются для создания бесконечных циклов или для тестирования в условных конструкциях.

6. Что означает строка `if test -f mans/i.$s`, встречаемая в командном файле?

Эта строка проверяет, существует ли файл. Путь содержит переменные `si` и `i`. Обратные слешы перед знаками доллара предотвращают интерпретацию `iis` как переменных, поэтому в имени искомого файла буквально содержатся символы `i.s`.

7. Объясните различие между конструкциями `while` и `until`.

Цикл `while` выполняется до тех пор, пока его условие истинно (код возврата равен нулю). Цикл `until` выполняется до тех пор, пока его условие ложно (код возврата не равен нулю), то есть он останавливается, когда условие становится истинным. `Until` является логической противоположностью `while`.